DealIQ

Revenue without guesswork.

An early-stage concept for an AI-powered deal clarity system built for B2B SaaS revenue teams.

Prepared for: Internal Review + Hackathon Submission | 2025

This document is a work in progress. Feedback is welcome.

A Note Before We Begin

This document is written for two audiences at once: a technical co-founder who will help decide what to build first, and hackathon judges who will evaluate whether the idea is worth pursuing.

We are not claiming DealIQ is finished, proven, or ready for market. We are claiming the problem is real, the gap in existing tools is genuine, and the approach we are proposing is worth building and testing.

Where we have data, we cite it. Where we are hypothesising, we say so. We think that honesty makes the idea stronger, not weaker.

The Problem We Are Trying to Solve

Sales leaders in B2B SaaS companies face a version of the same frustration at the end of every quarter: the pipeline looked healthy, the forecast said 80%, and the number still did not land. Not because the product was wrong. Not because pricing was off. But because something broke quietly inside the deal — and nobody noticed until it was too late.

That quiet breakdown usually has a pattern. A follow-up email sent with no next step defined. A discount offered out of anxiety rather than strategy. A key decision-maker who stopped responding three weeks ago but the deal is still marked active. A call where a rep verbally agreed to a non-standard term that never made it into the contract discussion.

These are not catastrophic failures. They are small behavioural leakages that compound across a team and a quarter. They are also almost entirely invisible to current tooling.

The core question DealIQ is trying to answer:

What if the health of a deal could be measured not just by activity logged in a CRM — but by the quality, structure, and consistency of the communication happening inside it?

Five Failure Modes We Have Observed

Failure Mode	What It Looks Like	Why It Is Hard to Catch
No next step defined	Rep sends a polite follow-up. Buyer stays friendly. Nothing moves.	CRM shows activity. Activity is not progress.
Emotional discounting	Deal goes quiet. Rep drops price to restart conversation.	No benchmark exists to flag this pattern.
Zombie pipeline	Deal is neither active nor dead. Forecast still includes it.	Reps are optimistic. Managers don't push hard enough.
Call-to-email mismatch	Rep commits something on a call that does not appear in the follow-up.	No one is cross-referencing transcripts and emails.
Legal drift	Rep verbally accepts a non-standard clause in negotiation email.	Legal is not watching every deal thread in real time.

Why Existing Tools Do Not Fully Solve This

We want to be honest here. The revenue intelligence space is not empty. Gong, Clari, People.ai, and Outreach are all serious, well-funded products. We are not dismissing them. We are identifying what they were built to do — and where that leaves a gap.

Tool	What It Does Well	What It Was Not Built For
Gong	Records calls, transcribes them, coaches reps post-interaction.	It observes and reports. It does not analyse whether what was said on a call matches what was written in a follow-up email.
Clari	Forecasting and pipeline visibility based on CRM signals.	It measures activity volume, not communication quality or structural discipline.
People.ai	Stakeholder tracking and activity capture automatically.	It tells you who was contacted. It does not score whether the interaction advanced the deal meaningfully.
Outreach	Engagement sequencing and workflow automation.	It helps reps send more emails faster. It does not evaluate whether those emails are moving deals forward.
Where DealIQ fits	Behavioural transparency at the deal communication level.	We are not trying to replace any of these. We are trying to sit between them — in the layer none of them deeply own.

An important caveat: we are an early concept. The above gaps are based on public documentation and our own analysis. We have not conducted formal competitor research at depth. This is our hypothesis — and we are open to being corrected.

What We Are Building

DealIQ is an attempt to build an AI layer that reads the communication happening inside deals — emails, call transcripts, contract threads — and surfaces patterns that indicate whether a deal is actually progressing or slowly dying.

The key word is transparency, not control. We are not building something that stops reps from doing things. We are building something that makes consequences visible before they become irreversible.

The design principle we are committed to:

If a rep is about to send an email with no next step, DealIQ does not block it.

It says: "This email reduces your deal health score by 7%. Suggested improvement: define next step before sending."

The rep decides. The consequence is visible. Behaviour changes over time — not because of enforcement, but because of awareness.

Three Pillars of the Product

Pillar	What It Does	Data Sources It Reads
1. Deal Reality Engine	Scores deal health based on communication quality, not just activity volume. Flags narrative mismatch between calls and emails. Detects stakeholder engagement gaps.	Email threads, call transcripts, CRM stage data, response time patterns
2. Commercial Guardrails	Monitors discount patterns against historical benchmarks. Flags repeated discount requests. Tracks contract clause deviations in negotiation emails.	Pricing history, deal notes, email negotiation threads, contract templates
3. Decision Forcing Layer	Identifies deals stuck in limbo. Surfaces an Advance / Close / Kill prompt with supporting evidence. Enforces next step definition before a deal can stay at its current stage.	CRM stage age, last meaningful interaction, next step field, stakeholder response rate

For the Technical Co-Founder

This section is written specifically for the person helping build this. The goal is not to prescribe architecture — that conversation should happen in person. The goal is to explain the feature logic clearly enough that we can make sensible decisions about what to build first.

Feature 1 — Deal Health Score

The core output of the product. A numerical score (0–100) assigned to each deal based on a weighted set of communication signals.

Signal	What We Are Measuring
Next step defined	Does the most recent outbound email contain an explicit next action with a date or timeframe?
Response recency	How many days since the buyer last responded meaningfully (not just an acknowledgement)?
Stakeholder depth	Has anyone beyond the first contact responded? Has the economic buyer ever engaged?
Interaction quality	Is the rep advancing the deal in each exchange, or just maintaining the relationship?
Discount pattern	Has a discount been mentioned more than once without a counter-proposal from the buyer?
Stage age	How long has this deal been at its current CRM stage relative to average deal velocity?

Note to tech co-founder: The weighting model for this score is something we should define together based on what data is cleanest to access first. We do not need all six signals in v1.

Feature 2 — Narrative Mismatch Detection

This is our sharpest differentiator. No current tool cross-references call transcripts against follow-up emails to detect inconsistencies.

The logic: after a call, the rep sends a follow-up email. DealIQ compares what was discussed in the call transcript against what appears in the email. If a commitment was made on the call — a pricing exception, a delivery timeline, a feature promise — and it does not appear in the email, a flag is raised.

Input	Detail
Call transcript	From Gong, Fireflies, or equivalent. Parsed for commitments, timelines, pricing mentions, and agreed next steps.

Follow-up email	The outbound email sent within 48 hours of the call. Parsed for the same categories.
Mismatch output	A plain-language flag: 'On the call you mentioned a 10% discount was possible. This does not appear in your follow-up email.'
Suggested action	DealIQ suggests adding the commitment explicitly or flagging it for manager review.

Note to tech co-founder: The NLP required here is achievable with current LLMs given a clear prompt structure. The main engineering question is the reliability of transcript parsing across different call recording platforms.

Feature 3 — Discount Escalation Heat Map

Tracks how many times a discount has been raised in a deal thread, by whom, and in what context. Compares against historical deal data to flag whether the current discount pattern is within normal range or indicates commercial pressure.

Input	Detail
Email thread analysis	Identify every mention of pricing, discount, reduction, or budget concern across the full thread.
Frequency threshold	If discount mentioned more than twice without a close signal, alert is triggered.
Historical benchmark	Compare against deals of similar size and stage. Is this discount pattern normal or an outlier?
Output	A heat map visual showing discount pressure over time within the deal, with a plain-language summary.

Feature 4 — Advance / Close / Kill

A decision-forcing surface for deals that have stalled. Rather than letting zombie deals sit in the pipeline, DealIQ surfaces a prompt after a configurable period of inactivity: this deal has not progressed in 18 days. Based on current signals, we recommend one of the following actions.

Option	What It Means
Advance	Rep commits to a specific next action within a defined window. Deal stays in pipeline.
Close	Rep marks deal as won. Evidence is logged.
Kill	Rep removes deal from active pipeline. Reason is captured for future learning.
Escalate	Rep flags deal for manager review. Useful when the rep is unsure or blocked by an external factor.

Note to tech co-founder: This is the simplest feature to build in v1. The logic is largely rules-based — time since last activity, stage age, and last meaningful response. No complex ML required for the first version.

The Adoption Question — Answered Honestly

One concern worth addressing directly: will a product positioned around deal governance actually get adopted, or will sales reps resist it?

The short answer is that adoption is a real risk — but not for the reason we initially assumed.

Research suggests that sales reps who use AI tools effectively are significantly more likely to hit quota than those who do not. The adoption problem is not that reps dislike tools. It is that most tools are too complex, too slow to show value, and too opaque in how they score or judge behaviour.

Our hypothesis on adoption:

Top-down: CROs and VP Sales adopt DealIQ because forecast accuracy and win rate are their accountability metrics. They mandate it.

Bottom-up: Reps adopt DealIQ because it helps them understand why their deals are stalling — not because it watches them.

The risk: If the deal health score is opaque or feels arbitrary, reps will ignore it. Score explainability is not a nice-to-have. It is the product.

What We Are Not Claiming

We think it is worth being explicit about the limits of this idea at this stage.

- We are not claiming DealIQ will work better than Gong or Clari in every dimension. It will not. Those are mature products with years of data.
- We are not claiming the deal health scoring model is proven. It is a hypothesis about which signals matter. That hypothesis needs to be tested with real data.
- We are not claiming this is the only gap in the market. There are likely others we have not identified.
- We are not claiming this is easy to build. Narrative mismatch detection in particular requires careful prompt engineering and reliable transcript data.

What we are claiming is more modest: the gap we have identified is real, existing tools do not deeply address it, and the approach we are proposing is technically feasible and worth testing.

What We Plan to Build for the Hackathon

Given time and resource constraints, our demo will focus on showing the concept clearly — not on full integration. The goal is to make the output so obvious that judges can understand the business impact without needing to see the engineering.

Will build for demo	Will defer to v1
Deal Health Score with simulated data	Live CRM integration (Salesforce / Zoho)
Narrative mismatch detection — one worked example	Multi-platform transcript parsing at scale
Discount Escalation Heat Map — visual output	Real-time email monitoring pipeline
Advance / Close / Kill decision surface	Multi-org data model and access controls
Simple dashboard showing all four features	Legal clause NLP at production scale

Note to judges:

We are showing a concept, not a finished product. We believe the clearest demonstration of value is a well-reasoned output — not the depth of integration behind it. We are happy to walk through the feature logic in detail during Q&A.

Technical Build Workflow — For the Co-Founder

This section is written as a direct brief for whoever is building the prototype. The goal is to get from zero to a working web app demo of the narrative mismatch feature within a single focused session, using whatever tools you are most comfortable with.

What we are building for the demo:

A web app with a split-screen interface. Left side: a call transcript. Right side: a draft follow-up email. A button labelled 'Check before sending.' When clicked, the AI compares both and surfaces any mismatches in plain language below.

No CRM integration needed. No browser extension needed. No live email required. Just clean inputs, smart comparison, clear output.

Step 1 — Prepare Your Sample Data First

Before writing a single line of code, prepare two text blocks you will use throughout testing. These simulate real inputs and let you iterate on the AI output without needing live integrations.

File	What to Write
Transcript (sample)	A realistic 300-400 word call transcript. Include a pricing mention (e.g. 'we could do 10% off if you sign before end of month'), a timeline commitment (e.g. 'onboarded within two weeks'), and an agreed next step (e.g. 'I will send a revised proposal by Thursday').
Email Draft (sample)	A follow-up email that intentionally omits one or two of those commitments. Keep it realistic — polite and professional. The omissions should not be obvious at first glance.

Why this matters: the quality of your sample data determines whether the demo is convincing. Spend 20 minutes here. It will save hours of debugging later.

Step 2 — The Core AI Prompt

This is the most important technical decision in the entire prototype. Everything depends on the prompt being precise. Below is a starting prompt you can paste directly into any LLM API or AI-assisted coding tool.

Prompt — paste this exactly as your system instruction:

You are a deal intelligence assistant. You will be given two inputs: a sales call transcript and a follow-up email draft that a sales rep is about to send.

Your job is to identify any of the following that appear in the transcript but are MISSING or INCONSISTENT in the email: (1) Pricing commitments or discount mentions. (2) Timeline or

delivery promises. (3) Agreed next steps with dates. (4) Feature or product promises. (5) Any other explicit commitment made to the buyer.

Return your response as a JSON object with two fields: 'mismatches' (an array of plain-language flags, each under 30 words, written as if speaking directly to the rep) and 'deal_health_impact' (a number between -1 and -20 representing score reduction). If no mismatches, return an empty array and 0.

Transcript: [INSERT TRANSCRIPT HERE]

Email Draft: [INSERT EMAIL DRAFT HERE]

The JSON structure is intentional. It lets the frontend display results cleanly without parsing free text. Keep the output structured even if you change the fields.

Step 3 — Choose Your Build Approach

Pick the approach that matches your speed and comfort. All three produce the same working demo. They are listed fastest to most flexible.

Option A — No-Code / Low-Code (n8n, Make, or Zapier)

Best if you want a working demo in under 3 hours without writing much code.

Step	What to Do
1. New workflow	In n8n or Make, start a new scenario. Use a manual trigger or webhook for the demo.
2. Two text inputs	One node for the transcript, one for the email draft. Hardcode the sample data for now — no live input needed yet.
3. OpenAI or Anthropic node	Add the LLM node. Paste the prompt from Step 2. Map transcript and email draft as variables into the prompt.
4. Parse the JSON output	Add a JSON parse step. Extract the mismatches array and deal_health_impact value as separate variables.
5. Display results	Route output to an HTML response or a Slack message. Clean text output is enough for the demo.
6. Add a simple UI	Use n8n's built-in form trigger or a free tool like Softr or Glide. Two text boxes, one button, results below. Done.

Option B — AI-Assisted Coding (Cursor, v0, or Replit)

Best if you are comfortable with code and want a cleaner, more impressive demo interface.

Step	What to Do

1. Scaffold the app	Open Cursor or Replit. Ask the AI to scaffold a simple Next.js or React app with two text areas and a submit button. v0.dev can generate the full UI from a one-line description.
2. Create one API route	A single backend endpoint — /api/check. It receives transcript and emailDraft as strings in the request body.
3. Call the LLM inside the route	Use the OpenAI or Anthropic SDK. Pass the prompt from Step 2 with inputs injected. Return the parsed JSON.
4. Display results	On submit, call your API. Show each mismatch as a warning card. Show the health score dropping visibly. Add a suggested fix under each flag.
5. Add sample data buttons	Pre-fill buttons for transcript and email draft. This makes the demo smooth and removes the risk of typing errors during the presentation.
6. Deploy	Vercel deploys a Next.js app in 60 seconds from a GitHub repo. Do this the night before.

Option C — Direct API (Most Control)

Best if your co-founder prefers working at the API level and wants full control over the logic.

Step	What to Do
1. One backend file	Flask or FastAPI in Python. Express in Node. Either works. One file is enough for the demo.
2. One POST endpoint	Accept transcript and email_draft as JSON fields. Validate they are non-empty strings.
3. Call the LLM directly	Use the Anthropic or OpenAI SDK. Set temperature to 0 for consistent, deterministic output. Pass the prompt from Step 2.
4. Parse and return	Extract JSON from the LLM response. Return it to the frontend. Add basic error handling for malformed JSON — it will happen occasionally.
5. Minimal HTML frontend	A single index.html file. Two textareas, one button, one results div. Fetch the API on click. Render results inline. No framework needed.
6. Host it	Replit hosts and runs the app in the same environment. Railway or Render also work in under 5 minutes from a repo.

Step 4 — The Four Demo Screens

Regardless of which approach you build with, the demo needs to show four moments clearly. Prepare these as distinct UI states so you can click through them smoothly during the presentation without live typing.

Screen	What It Shows

Screen 1 — The Deal	A simple deal card at the top. Deal name, company, stage, current health score (e.g. 74 out of 100). This gives judges context before anything runs.
Screen 2 — The Inputs	Split view. Left: call transcript pre-loaded. Right: email draft pre-loaded. The 'Check before sending' button prominent between them.
Screen 3 — The Mismatch Result	After clicking: two warning cards appear. Each states the mismatch in plain language. The deal score updates visibly downward. A suggested fix appears under each flag.
Screen 4 — The Rep Decision	Three action buttons: 'Add to email', 'Intentionally omit', 'Flag for manager'. Rep clicks one. The action is logged. Score either recovers or stays reduced. This closes the governance loop.

Recommended Tech Stack

Layer	Recommended Choice and Why
Frontend	Next.js or plain React. Use v0.dev to generate the UI layout from a single text description — it produces clean Tailwind components instantly.
Backend	Next.js API routes keeps everything in one repo. FastAPI if Python is faster for you personally.
AI Model	Claude (Anthropic) or GPT-4o (OpenAI). Both handle structured JSON output well. Claude follows formatting instructions more consistently in our testing.
Hosting	Vercel for Next.js — free tier, deploys in 60 seconds from GitHub. Replit if you want everything in one environment without a separate deployment step.
Sample Data	Hardcode two or three deal scenarios. Label them by type — SaaS deal, Enterprise deal, Renewal deal. Let judges pick which to run. This adds realism without integration risk.
No-code alternative	n8n cloud with a Softr or Glide frontend. The entire prototype is buildable without a code editor if that is faster.

The one thing that will make or break the demo:

The mismatch output must be in plain, human language. Not 'Semantic inconsistency detected between utterance 3 and paragraph 2.' But: 'On the call you offered a 10% discount before end of month. Your email does not mention this.'

Spend as much time tuning the output language of the prompt as you spend on the interface. Judges will read those words out loud. Make them land.

"Revenue without guesswork."

That is what we are working toward. We know we are not there yet.